

UNSW-NB15 Feature Derivability from Raw Packet Dataset

Methodology

UNSW-NB15 is a **flow-based** dataset (built by UNSW using Argus/Bro + 12 custom scripts). Your dataset is **raw packet-level** data. To derive UNSW-style features, packets must first be grouped into flows.

Flow key: (ip_src, ip_dst, t_sport, t_dport, ip_proto), with a timeout to close idle flows (e.g., 120s TCP idle, or explicit close on FIN-ACK/RST). "Source" = the IP that sent the first packet of the flow (SYN sender for TCP).

Once flows are constructed, features fall into four categories based on how reliably they can be computed:

- 1. **Fully derivable** — direct aggregation over packets in a flow, no ambiguity
- 2. **Approximately derivable** — requires your own heuristic/logic, won't exactly match Argus's internal algorithm
- 3. **Cross-flow derivable** — needs a second pass over the completed flow table (time-windowed counts)
- 4. **Not feasible** — requires payload inspection or external ground truth you don't have

1. Fully Derivable (Direct Aggregation)

These come straight from grouping packets by flow key and aggregating fields already in your schema.

UNSW Column	Derivation
srcip/dstip	ip_src/ip_dst of the flow key
sport/dsport	t_sport/t_dport of the flow key
proto	Map ip_proto(IANA protocol number) to name — 6=tcp, 17=udp, 1=icmp, etc.

dur	max(ts) - min(ts) across all packets in the flow
Stime/Ltime	min(ts)/max(ts) in the flow
sbytes/dbytes	Sum of ip_totallen for packets in the src → dst direction / dst → src direction
Spkts/Dpkts	Packet count per direction
smeansz/ dmeansz	Mean of ip_totallen per direction
Sload/Dload	(bytes in that direction × 8) / dur— bits per second
Sjit/Djit	Standard deviation of consecutive inter-arrival times (ts diffs) per direction
Sintpkt/Dintpkt	Mean inter-arrival time per direction
sttl/dttl	ip_ttl from the first packet in that direction (mode across the flow also acceptable)
swin/dwin	t_window from the first SYN (src side) / SYN-ACK (dst side) packet
stcpb/dtcpb	Initial sequence number: t_seqnum from SYN (src) / SYN-ACK (dst)
is_sm_ips_ports	1 if ip_src == ip_dst and t_sport == t_dport, else 0
tcprrt	synack + ackdat
synack	ts(SYN-ACK) - ts(SYN)
ackdat	ts(ACK) - ts(SYN-ACK)

Count: 30 columns (srcip, dstip, sport, dsport, proto, dur, Stime, Ltime, sbytes, dbytes, Spkts, Dpkts, smeanz, dmeanz, Sload, Dload, Sjit, Djit, Sintpkt, Dintpkt, sttl, dttl, swin, dwin, stcpb, dtcpb, is_sm_ips_ports, tcprrt, synack, ackdat), computable purely from within-flow packet aggregation.

2. Approximately Derivable (Heuristic-Based)

These are computable, but require you to design your own logic. They will **not exactly match** UNSW's original Argus output since the internal state machines/lookup tables aren't public.

UNSW Column	Method	Caveat
state	Build a TCP state machine from the t_flags sequence per flow: SYN → SYN/ACK → ACK =CON; FIN seen =FIN; RST seen =RST; incomplete handshake =INT/REQ	This is your own labeling scheme — Argus's exact state code table isn't replicable without its source

service	Port-to-service lookup table: 80/8080 http, 21 ftp 22 ssh 25 smtp 53 dnsetc.	No payload inspection available, so non-standard port usage (e.g., HTTP on a random port) will be mislabeled
sloss/ dloss	Detect retransmissions per direction via duplicate or out-of-order t_seqnum values; count occurrences	Approximate — true loss detection normally needs ACK-based confirmation, which is harder without full payload/ack tracking

Count: 4 columns (state, service, sloss, dloss).

3. Cross-Flow Derivable (Second Pass Required)

These are historical/contextual counts that depend on **other flows**, not just the current one. They require building the full flow table first, sorting by time, then sliding a window (UNSW typically uses the last 100 flows) to count matches.

UNSW Column	Definition	Depends On
ct_srv_src	Count of prior flows (in window) sharing service+ source IP	service(approximate)
ct_srv_dst	Count of prior flows sharing service+ destination IP	service(approximate)
ct_dst_ltm	Count of prior flows sharing destination IP	—
ct_src_ltm	Count of prior flows sharing source IP	—
ct_src_dport_ltm	Count of prior flows sharing source IP + destination port	—
ct_dst_sport_ltm	Count of prior flows sharing destination IP + source port	—
ct_dst_src_ltm	Count of prior flows sharing source IP + destination IP	—
ct_state_ttl	Count of prior flows sharing (state, sttl, dttl) bucket	state(approximate)

Count: 8 columns. Mechanically derivable, but three of them inherit error from the approximate service/state features above — so their accuracy is bounded by those.

4. Not Feasible (No Payload / No Ground Truth)

These cannot be derived from your schema at all — not approximately, not partially. Your raw columns cover IP/TCP headers and connection metadata only (ip_*, t_*, o_* option fields); there is no payload/application-layer field anywhere in the schema, and no attack-labeling field. That's the hard blocker for every column below.

4a. Blocked — require payload/application-layer inspection

UNSW Column	Reason it can't be derived
trans_depth	This is the pipelined request depth within an HTTP connection (e.g., 1st, 2nd request on a keep-alive connection). It's only visible by parsing the HTTP request line inside the payload. Your schema has no payload field (o_* columns are TCP options like MSS/SACK/window-scale — not application data), so there's nothing to parse this from.
res_bdy_len	The uncompressed size of the HTTP response body. This is a value read from HTTP headers/content inside the payload after decompression — packet/IP-header fields like ip_totallen only give you total packet size, not the parsed body length. No payload = no way to compute it.
ct_flw_http_mthd	Counts flows using HTTP methods GET/POST. Requires reading the HTTP request line (GET /path HTTP/1.1) from payload bytes. Your dataset stops at the transport layer, so the HTTP method never appears anywhere in your columns.
is_ftp_login	Flags whether an FTP session performed a USER/PASS authentication exchange. FTP control commands travel in the payload of the FTP control channel (port 21) — there's no header-level signal for this, so it's undetectable from ip_*/t_* fields alone.
ct_ftp_cmd	Counts FTP commands issued in a flow (RETR,STOR,LIST, etc.). Same as above — these are ASCII text lines inside the payload, not header metadata.

4b. Blocked — require external ground truth (not a traffic feature at all)

UNSW Column	Reason it can't be derived
-------------	----------------------------

attack_cat	This is a human/analyst-assigned label naming which attack family a flow belongs to (DoS, Exploits, Fuzzers, etc.), assigned by UNSW researchers who controlled the attack traffic generation in a testbed. There is no packet-header signature that reveals "this is a Fuzzer attack" – it's knowledge external to the traffic itself. You'd need your own attack testbed with known ground truth, or a labeled reference dataset, to reproduce this.
Label	Binary normal(0)/attack(1) tag, assigned the same way as attack_cat— by knowing in advance which flows were intentionally malicious during data collection. Passive raw packet capture alone can never reconstruct "was this traffic intentionally an attack," since malicious and benign traffic use the exact same protocol fields.

Count: 7 columns (trans_depth, res_bdy_len, ct_flw_http_mthd, is_ftp_login, ct_ftp_cmd, attack_cat, Label) – 5 blocked by missing payload, 2 blocked by missing ground-truth labels.

Summary

Category	Count	Notes
Fully derivable	30	Direct packet aggregation, no ambiguity
Approximately derivable	4	Your own heuristics, won't match Argus exactly
Cross-flow derivable	8	Needs second pass, 3 inherit approximation error from state/service
Not feasible	7	5 need payload inspection, 2 need external ground-truth labels
Total	49	All UNSW-NB15 columns accounted for

Practical takeaway: You can reliably reconstruct **34 of 49** UNSW-NB15 features (30 exact + 4 close-heuristic) directly from packet headers, and a further 8 (cross-flow counts) on top of that – though 3 of those 8 inherit some error from the heuristic state/service fields. The remaining 7 are hard blockers: 5 need payload/application-layer data your capture doesn't include, and 2 (attack_cat, Label) need external ground truth from a controlled attack testbed – no amount of clever feature engineering recovers those from passive header data alone.